

A bicriterion approach to time/cost trade-offs in sequencing

Luk N. VAN Wassenhove

Catholic University of Leuven, Belgium

Kenneth R. BAKER

Dartmouth College, Hanover, NH, U.S.A.

This paper discusses a bicriterion approach to sequencing with time/cost trade-offs. This approach, which produces an efficient frontier of possible schedules, has the advantage that it does not require the sequencing criteria to be measurable in the same units as the resource allocation cost. The basic single-machine model is used to treat a class of problems in which the sequencing objective is to minimize the maximum completion penalty. It is further assumed that resource allocation costs can be represented by a linear time/cost function.

1. Introduction

Sequencing problems comprise a special class of scheduling problems in which the only decisions involve the order in which tasks are to be completed. More generally, scheduling problems involve resource allocation as well as sequencing, but this combination is usually very difficult to analyze.

One of the simplest and most basic forms of resource allocation is represented by the time/cost trade-off. In a time/cost model, the application of additional resources (as measured by their cost) reduces the time required to carry out a task. Time/cost models have seldom been examined in conjunction with sequencing decisions. Vickson [5] has analyzed a model in which time/cost considerations are superimposed on the problem of minimizing the total delay cost. Elmaghraby and Pulat [2] have analyzed a similar model in which time/cost considerations are superimposed on the problem of minimizing the total tardiness cost in the presence of precedence restrictions. In both of these approaches, the cost incurred as a result of sequencing and the cost incurred as a result of

resource allocation are combined into a one-dimensional criterion.

In this paper an approach to sequencing with time/cost trade-offs is described which distinguishes between sequencing criteria and resource allocation costs. This approach gives rise to a bicriterion formulation of the problem, in the spirit of Van Wassenhove and Gelders [4], who first discussed the principles of sequencing with multiple criteria. One of the main advantages of this approach is that it does not require that sequencing criteria be measurable in the same units as the resource allocation cost. This feature is particularly suited to situations in which the sequencing objective takes a minimax form, such as minimizing the maximum lateness.

The basic single-machine model is used to treat a class of problems in which the sequencing objective is to minimize the maximum completion penalty. It is assumed that resource allocation costs can be represented by a linear time/cost function, and that the scheduling decision will ultimately be based on information about efficient combinations of completion penalties and resource allocation costs. The solution algorithm therefore produces an efficient frontier of possible schedules, as a basis for the final selection.

2. The bicriterion: maximum tardiness, crashing costs

Consider the problem of constructing efficient sequences for the bicriterion of maximum tardiness and crashing costs.

Let

t_j : actual processing time, $0 < a_j \leq t_j \leq b_j$,

d_j : due date,

$c_j(b_j - t_j)$: crashing cost,

C_j : completion time,

$T_j = \max\{0, C_j - d_j\}$: tardiness,

$T_{\max} = \max_j \{T_j\}$: maximum tardiness.

The bicriterion to be considered is the pair (G, K) where

$$G = T_{\max}, \quad K = \sum_{j=1}^n c_j(b_j - t_j).$$

A sequence π^0 with a bicriterion pair (G^0, K^0) is efficient if there is no other sequence π such that

$$G \leq G^0, \quad K \leq K^0$$

with at least one strict inequality.

Note that, when the t_j are given (i.e., K is given), then G is minimized by the earliest due date rule (EDD-rule) [1]. It follows that any efficient sequence should satisfy the EDD-rule.

It is further assumed that the jobs are to be indexed and sequenced in EDD-order. The following algorithm constructs the efficient frontier.

A greedy algorithm.

(1) Set $t_j = b_j$ for all jobs and identify job k such that

$$k = \min\{j | T_j = T_{\max}\}.$$

(2) Find job i such that $c_i = \min\{c_j | j \leq k, t_j > a_j\}$. If no such job exists, stop.

(3) Crash job i (T_k decreases) until one of the following occurs:

(a) $T_k = 0$: stop;

(b) $t_i = a_i$: return to (2);

(c) $T_k = T_r$, where $r = \min\{j | T_j = T_k, j < i\}$.

Let $k \leftarrow r$ and return to (2).

Example. Consider the job data given in Table 1. Apply the greedy algorithm.

(1) The EDD-sequence with all $t_j = b_j$:

	1	2	3	4	5	6	
C_j	6	14	19	24	31	37	t
T_j	1	6	7	6	10	9	

$k = 5$.

Table 1

	1	2	3	4	5	6
a_i	4	5	4	1	6	4
b_i	6	8	5	5	7	6
c_i	3	4	5	2	10	6
d_i	5	8	12	18	21	28

The first efficient point is $(G_1, K_1) = (10, 0)$.

(2) $i = 4$.

(3) Crash job 4 by 3 units. Then $T_5 = T_4 = 7$, $k = 3$. The second efficient point is $(G_2, K_2) = (7, 6)$.

(2) $i = 1$.

(3) Crash job 1 by 2 units. Then $t_1 = a_1 = 4$. The third efficient point is $(G_3, K_3) = (5, 12)$.

(2) $i = 2$.

(3) Crash job 2 by 3 units. Then $t_2 = a_2 = 5$. The fourth efficient point is $(G_4, K_4) = (2, 24)$.

(2) $i = 3$.

(3) Crash job 3 by 1 unit. Then $t_3 = a_3 = 4$. The fifth efficient point is $(G_5, K_5) = (1, 29)$.

(2) Stop.

Fig. 1 shows the efficient frontier in the objective function space.

The efficient frontier is obtained by connecting the efficient (corner) points given by the greedy algorithm. All the points on this piecewise linear curve are efficient. The advantage of the greedy algorithm is that it only computes the breakpoints.

Theorem. The greedy algorithm constructs the efficient frontier.

Clearly, there exists an efficient point for every feasible value of G , i.e., $T_{\min} \leq G \leq T_{\max}$, where $T_{\max}$ and $T_{\min}$ correspond to the maximum tardiness when the jobs have processing times b_j and a_j respectively (the jobs are in EDD-order).

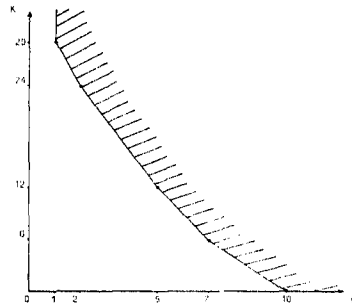


Fig. 1. Efficient frontier for the example.

Let $x_i = b_i - t_i$ (the amount of crashing of job j). Then, an efficient point is given by the solution to the following LP problem:

$$\begin{aligned} \text{Min } K &= \sum_{j=1}^n c_j x_j, \\ \text{s.t. } G &\geq \sum_{i=1}^j (b_i - x_i) - d_j, \quad \text{all } j, \\ 0 &\leq x_j \leq b_j - a_j, \quad \text{all } j. \end{aligned}$$

This formulation is identical to that given by Vickson [6] except that in [6] the objective function is the sum of maximum tardiness and crashing costs.

If $L_j = \sum_{i=1}^j b_i - d_j$ (the lateness of job j when no job is crashed), then the tardiness related constraints become

$$\sum_{i=1}^j x_i + G \geq L_j, \quad \text{all } j.$$

Any constraint with $L_j \leq 0$ or $L_j \geq L_j$ for some $k < j$ is redundant. Vickson [6] shows that the remaining problem is equivalent to an easily solvable production-inventory problem, the solution of which is in turn identical to the one given by the greedy algorithm (for that specific value of G).

Note that if one starts with $G = T_{\min}$ and gradually increases G in order to find the other efficient points, the only change in the LP problem is that additional constraints may become redundant and hence may be deleted. Vickson's algorithm is identical to our greedy algorithm except that it finds only one efficient point (the one that minimizes the sum of maximum tardiness and crashing costs). Vickson also shows that in the worst case his algorithm runs in $O(n^3)$, [6].

The reader should convince himself that our greedy algorithm finds all efficient points in at most $O(n^2)$ time. This follows because the algorithm crashes any job at most once (which gives the corner points of the efficient frontier). The entire piecewise linear and convex frontier can be obtained by connecting the corner points since the crashing cost increases at a constant rate (c_i) until either condition 3(b) or 3(c) occurs. At this point the rate increases (assuming $c_i \neq c_j$ for $i \neq j$).

It is the simplicity of the linear crashing cost function (K) for the jobs and of the maximum tardiness objective (G) that leads to the $O(n^2)$

greedy algorithm and the piecewise linear (and convex) efficient frontier in the (G, K) objective function space. To illustrate this point, consider the maximum weighted flowtime objective, i.e., $G = \max_j \{w_j C_j\}$. For given t_j , G is minimized by ordering the jobs in non-increasing order of w_j [1]. Hence, all efficient points for the (G, K) bicriterion correspond to this sequence. The job sets to be considered in step (2) of the greedy algorithm are no longer decreasing. It is possible that the same job has to be reconsidered for crashing several times. Therefore the $O(n^2)$ result no longer holds. In the next section a general procedure is given for the class of sequencing objectives which contains $\max_j \{w_j C_j\}$, among others.

Observe that for the sequencing objectives discussed so far, the set of efficient points corresponds to a unique sequence. It is clear that whenever the sequence can change (which will happen for all but very simple sequencing objectives), it will become very hard, if not impossible, to find a polynomial algorithm that generates the efficient frontier.

3. A more general class of sequencing cost functions

It is possible to treat a more general class of sequencing cost functions in an analogous fashion. Suppose that job j is characterized by a cost function $g_j(t)$, which specifies the cost incurred when job j is completed at time t . It is assumed that the functions $g_j(t)$ are nondecreasing in t and also that they satisfy the following condition: there exists an ordering of the jobs such that

$$g_1(t) \geq g_2(t) \geq \dots \geq g_n(t) \quad \text{for all } t \geq 0. \quad (3.1)$$

This condition does not include arbitrary nondecreasing functions, but it does include some special cases of interest. For example:

(a) $g_j(t) = \max(0, t - d_j)$. Here the cost function is the tardiness of job j , giving rise to the special case considered in the previous section.

(b) $g_j(t) = w_j t$. Here the cost function is the weighted completion time (weighted flowtime) of job j .

(c) $g_j(t) = w_j \max(t - d_j)$. Here the cost function is the weighted tardiness of job j , but all jobs are assumed to have the same due-date.

When (3.1) holds, it is possible to find a single

job sequence that will minimize $\max_j \{g_j(t)\}$ for any selection of processing times. In particular, the desired sequence for (3.1) is 1, 2, ..., n .

The bicriterion to be considered is the pair (G, K) where

$$G = \max_j \{g_j(C_j)\} \quad \text{and} \quad K = \sum_{j=1}^n c_j(b_j - t_j).$$

Thus the first criterion is the maximum completion cost and the second criterion is the total crashing cost.

The algorithm that generates an efficient point for this bicriterion consists of three procedures, which are implemented in series. Procedure 1 establishes the job sequence and the extreme values of G ; Procedure 2 computes the amount of job crashing that must be scheduled; and Procedure 3 determines the optimal crashing allocations.

Procedure 1, due to Lawler [3], is a general algorithm for finding the job sequence that minimizes the maximum cost. When applied to the n -job problem with $t_i = b_i$, the procedure yields a job sequence that will be optimal at all points on the efficient frontier. Furthermore, the same calculations yield the maximum value of G (denoted $G_{\max}$) on the efficient frontier.

Procedure 1.

- (1) Set $M = \Sigma_{j=1}^n t_j$.
- (2) Find i such that $g_i(M) = \min \{g_j(M) \mid j \text{ unscheduled}\}$.
- (3) Schedule job i to be completed at time M (i.e., in the last available sequence position). Subtract t_i from M .
- (4) Repeat step (2) until all jobs are scheduled.

By putting all $t_i = a_i$ in the sequence obtained by Procedure 1, $G_{\min}$, the minimum value of G , can easily be found.

Given a specific value of G , Procedure 2 calculates the amount of crashing that will be required in various portions of the schedule. The job sequence has already been determined by Procedure 1 and it is assumed that the jobs are numbered according to their position in the sequence.

Procedure 2.

- (1) Set $j = 1$. Let $D_0 = 0$ and $u_0 = 0$.
- (2) Calculate u_j , defined as the largest t for which $g_j(t) = G$.

5

- (3) Set $D_j = \max(0, \Sigma_{i=1}^j b_i - \Sigma_{i=0}^{j-1} D_i - u_j)$.
- (4) Increment j and repeat steps (2) and (3) until D_n is calculated.

The quantity D_j is the *incremental* crashing requirement for job j . That is, the amount of crashing necessary among the first j jobs, so that job j will be completed no later than time u_j and therefore satisfy $g_j(C_j) \leq G$. D_j is 'incremental' in the sense that it recognizes the crashing requirements already imposed by earlier jobs in the sequence. Suppose that job j was not crashed. Then its completion time would be

$$C_j = \sum_{i=1}^j b_i - \sum_{i=0}^{j-1} D_i.$$

If $C_j \leq u_j$, then the cost associated with job j will not exceed G , and so there will be no incremental crashing requirement. If $C_j > u_j$, then the completion of job j must be advanced by $C_j - u_j$ in order that its completion cost does not exceed G . Thus we have:

$$\text{if } \sum_{i=1}^j b_i - \sum_{i=0}^{j-1} D_i \leq u_j, \\ \text{then } D_j = 0;$$

$$\text{if } \sum_{i=1}^j b_i - \sum_{i=0}^{j-1} D_i > u_j, \\ \text{then } D_j = \sum_{i=1}^j b_i - \sum_{i=0}^{j-1} D_i - u_j.$$

Therefore this calculation reduces to the formula in step (3) of Procedure 2.

The incremental crashing requirements D_i describe a special pattern of crashing needs. Specifically, job crashing (in an amount totalling D_i) must be assigned in some pattern among jobs 1, 2, ..., j , in addition to the requirements $D_1, D_2, \dots, D_{j-1}$. It may well be intuitive at this point that, in the presence of linear crashing costs, the assignment of the quantity D_i can also be made without regard for the requirements $D_1, \dots, D_{j-1}$. This fact is exploited in Procedure 3, which considers the incremental requirements in job order and simply finds the cheapest assignment available to meet the need.

Procedure 3.

- (1) Set $t_1 = b_1 - D_1$. Set $K = c_1 D_1$. Let $j = 2$.

- (2) If $D_j = 0$, go to (4); otherwise proceed.
- (3) For job j , find $i(j)$ such that $c_{i(j)} = \min\{c_k | k \leq j \text{ and } t_k > a_k\}^1$.
 - (a) If $D_j \leq t_{i(j)} - a_{i(j)}$, then: reduce $t_{i(j)}$ by D_j , increase K by $c_{i(j)}D_j$, go to (4).
 - (b) If $D_j > t_{i(j)} - a_{i(j)}$, then: reduce D_j by $t_{i(j)} - a_{i(j)}$, increase K by $c_{i(j)}(t_{i(j)} - a_{i(j)})$, set $t_{i(j)} = a_{i(j)}$, repeat (3).
- (4) Increment j . If $j > n$ stop; otherwise go to (2).

Procedure 3 is a greedy algorithm, which meets each requirement D_j in turn, with the cheapest existing alternative and which makes no subsequent reduction of any crashing quantity assigned during step (3). At its completion, Procedure 3 yields the minimum cost, K , corresponding to the given value of G . The pair (G, K) lies on the efficient frontier.

The entire efficient frontier can be generated as follows. First, execute Procedure 1 to find $G_{\min}$ and $G_{\max}$. The point $(G_{\max}, 0)$ identifies one extreme of the frontier. Then let G be reduced from $G_{\max}$ to $G_{\min}$ in steps of a given size, each time repeating Procedures 2 and 3. (To re-initialize the algorithm each time G is decreased, it is sufficient to reset $b_j = t_j$, where t_j is the processing time assigned to job j at the end of the previous iteration.) This algorithm will produce efficient combinations of G and K on a grid of any desired refinement, depending on the selection of the step size for decrementing G .

3.1. Justification of the solution algorithm

The validity of the algorithm is demonstrated by an induction argument. The algorithm clearly provides an optimal solution to any one-job problem. Hence it is assumed that the algorithm is optimal for any problem containing $n-1$ jobs (where $n > 1$), and it is then proved that the algorithm is optimal for any n -job problem.

The first requirement in an n -job problem is that job n must be completed no later than time u_n , where u_n is given by $g_n(u_n) = G$. This is simply a feasibility condition, and it can be seen that this is guaranteed by Procedure 2, in the calculation of D_n . It now remains to show that the steps followed

in Procedure 3 with respect to job n construct an optimal solution. Note that Procedure 3 does not consider the requirements of job n until it has produced a schedule for the first $n-1$ jobs. By the induction argument, this portion of the schedule is optimal.

Let M_{n-1} denote the completion time of job $n-1$ in the schedule constructed for the first $n-1$ jobs. The following cases may arise:

Case 1. $M_{n-1} \leq u_n - b_n$;

Case 2. $M_{n-1} > u_n - b_n$;

Subcase 2(a). $c_n \leq c_j$ for all $j < n$;

Subcase 2(b). $c_i < c_n$ for some $i < n$, where $t_i > a_i$.

In case 1 there is no incremental crashing requirement for job n and thus $D_n = 0$, and an optimal solution consists of setting $t_n = b_n$ and constructing an optimal schedule for the first $n-1$ jobs. It follows that Procedure 3 is optimal for the n -job problem in this case.

In case 2 job n imposes an incremental crashing requirement, $D_n > 0$. However, in subcase 2(a), job n is the cheapest job to crash and thus an optimal solution consists of crashing job n by an amount D_n (if this is possible) and then, separately, finding an optimal schedule for the first $n-1$ jobs. This decomposition occurs because the crashing of job n has no effect on the scheduling of earlier jobs. It follows that Procedure 3 is optimal for the n -job problem in this case, because it constructs a schedule consisting of the optimal schedule for the first $n-1$ jobs, followed by job n crashed by an amount D_n . If job n cannot be crashed sufficiently far (i.e., $D_n > b_n - a_n$), then the optimal solution is to set $t_n = a_n$ (as Procedure 3 does) and then to proceed as in subcase 2(b).

In subcase 2(b) the optimal schedule for the first $n-1$ jobs will not appear intact as part of the solution to the n -job problem. The cheapest way to permit job n to be completed at time u_n corresponds to some modification of the optimal schedule for the first $n-1$ jobs; in particular, a modification in which job $n-1$ is completed earlier than at time M_{n-1} . Under the conditions of subcase 2(b), an optimal solution to the n -job problem can be viewed as arising from the following decomposition: (D1) set $t_n = b_n$ and let job n be completed at time u_n ; (D2) obtain the optimal solution to the remaining $(n-1)$ -job problem under the constraint that job $n-1$ is completed at time $u_n - b_n$. However, an optimal solution for the constrained subproblem is equivalent to an optimal

¹ If no such job exists, the pair (G, K) is infeasible.

solution to an unconstrained problem in which the $(n-1)$ cost functions are modified in a special way. Let primes denote the modified cost functions. For $j = 1, 2, \dots, n-1$ set

$$g'_j(t) = \begin{cases} g_j(t), & \text{for } t \leq u_n - b_n, \\ \infty, & \text{for } t > u_n - b_n. \end{cases}$$

Then the optimal schedule for jobs 1, 2, ..., $n-1$ with cost functions $g'_j(t)$ is also an optimal schedule for the same jobs with their original cost functions and the requirement that job $n-1$ must be completed no later than $u_n - b_n$. Furthermore, by hypothesis, Procedure 3 finds an optimal schedule for the subproblem in (D2).

Now that an optimal solution to the n -job problem can be recognized it remains only to verify that this is produced by Procedure 3. It can easily be seen that when Procedure 3 is applied to the n -job (original) problem, the same schedule is constructed as would result from the decomposition (D1, D2) described above as the optimal solution. Therefore, Procedure 3 constructs an optimal schedule for the n -job problem in all cases, and the induction argument is complete.

3.2. Computational requirements of the algorithm

In order to generate the efficient frontier using this solution algorithm, Procedure 1 must be executed once and Procedures 2 and 3 must be executed for each value of G from $G_{\min}$ to $G_{\max}$. In general, the different G -values must be given, possibly by specifying the step size between adjacent G -values.

The computational requirement is simplified considerably if the functions $g_j(t)$ are linear. In this case it is not always necessary to repeat Procedures 2 and 3 for each G -value. Instead, information in one application of Procedures 2 and 3 can be utilized in order to identify the efficient frontier over a range of G -values.

Suppose that Procedures 2 and 3 are applied to the problem and that the resulting efficient point produced is (G_0, K_0) . Suppose next that the value of G is reduced slightly, by an amount ΔG . Then for each job j with $D_j > 0$ there will be a need to assign additional crashing somewhere in the schedule. Moreover, the amount of crashing to be assigned will be proportional to ΔG due to the linearity of the functions $g_j(t)$.

Let $Q = \{j | D_j > 0\}$ denote the set of (critical) jobs for which an additional crashing requirement results from a small reduction in G . For each job j in the set Q let $i(j)$ denote the index of the job that would be crashed in order to meet the additional requirement in D_j induced by the change ΔG . As in Procedure 3, $i(j)$ is defined by

$$c_{i(j)} = \min \{c_k | k \leq j \text{ and } r_k > a_k\}.$$

The marginal change ΔG will result in an increase in crashing cost given by

$$\Delta K = \left(\sum_{j \in Q} v_j c_{i(j)} \right) \Delta G \quad (3.2)$$

where v_j denotes the appropriate proportionality constant reflecting the linear relation of time to cost expressed in the $g_j(t)$ function.

Thus it is possible to calculate the slope along the efficient frontier as G is reduced. In addition, this slope will hold until there is a change in the structure of the crashing assignments. Two types of changes can affect this structure:

(1) A job's processing time may be compressed to its limit ($t_i = a_i$) in the execution of Procedure 3. Thus in step (3) (a) of Procedure 3 any further change ΔG , which would cause such a limit to be reached, can be calculated. The lowest of these values, denoted by ΔG_1 , can be recorded as Procedure 3 is executed.

(2) A new job can become critical, by virtue of the condition $D_j > 0$. Thus each time $v_j = 0$ in step (3) of Procedure 2 any further change ΔG , which would cause D_j to become positive, can be calculated. The lowest of these values, denoted by ΔG_2 , can be recorded as Procedure 2 is executed.

The structure of crashing assignments will be altered if G is reduced more than $\Delta G = \min(\Delta G_1, \Delta G_2)$. Points at which this structural change occurs correspond to 'corner' points on the efficient frontier. In the interval between corner points, the behavior of the frontier is given by (3.2). From any corner point, the calculation of $\min(\Delta G_1, \Delta G_2)$ will show where the next corner point will occur as G is reduced. Thus it is only necessary to repeat Procedures 2 and 3 for the corner points on the frontier. The number of corner points is no greater than $2n-1$, as the discussion above indicates. The computational effort involved in Procedure 2 is of order n , while the effort involved in Procedure 3 is of order n^2 (i.e., for finding the minimal cost

among crashing alternatives for each of n jobs). Since these procedures are repeated a number of times no more than order n , and since Procedure 1 requires an effort no greater than order n^2 , the entire efficient frontier can be generated, in the linear case, with a computational effort that is order n^3 in the worst case. In addition, if $g_j(t)$ is piecewise linear with two different slopes, as is the case when linear tardiness penalties apply, the algorithm still has a worst-case computational requirement of order n^3 .

Example. Consider the bicriterion problem $\{\max w_j C_j, \sum_j C_j(b_j - t_j)\}$ with the data given in Table 2.

(a) Find the efficient point corresponding to $G = 18$. Procedure 1 yields:

	3	1	2	4
$C_j =$	3	7	13	18
$w_j C_j =$	12	21	26	18

Procedure 2 yields:

$$u_1 = 4.5, u_2 = 6, u_3 = 9, u_4 = 18.$$

(The jobs have been re-indexed according to the sequence 3-1-2-4)

$$D_1 = 0, D_2 = 1, D_3 = 3, D_4 = 0.$$

Procedure 3:

- (1) $t_1 = 3, K = 0, j = 2$,
- (2) $D_2 \neq 0$,
- (3) $i(2) = 1, D_2 < t_1 - a_1 = 3 - 1 = 2$,
 $t_1 = 2, K = 4$,
- (4) $j = 3$,
- (2) $D_3 \neq 0$,
- (3) $i(3) = 3, D_3 < t_3 - a_3 = 6 - 3 = 3, t_3 = 3$,
 $K = 4 + 6 = 10$,
- (4) $j = 4$,
- (2) $D_4 = 0$,
- (4) Stop.

Table 2

j	1	2	3	4
b_j	4	6	3	5
a_j	2	3	1	2
w_j	3	2	4	1
c_j	5	2	4	1

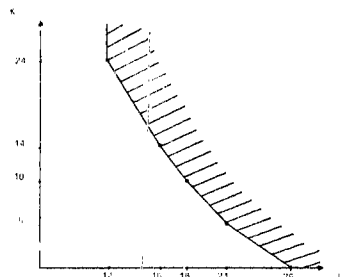


Fig. 2. Efficient frontier for the example.

The efficient point is (18, 10).

(b) Find the efficient frontier.

$$G_{\max} = 26, \quad G_{\min} = 12$$

The first efficient corner point is $(G_1, K_1) = (26, 0)$.

Letting G decrease, it can be seen that job 3 will become critical at $G = 21$. The corresponding corner point is $(G_2, K_2) = (21, 5)$.

The next corner point corresponds to job 3 being compressed to its limit: $(G_3, K_3) = (18, 10)$. Finally, $(G_4, K_4) = (16, 14)$ and $(G_5, K_5) = (12, 24)$ correspond to jobs 1 and 2 reaching their maximum compression. Fig. 2 shows the efficient frontier.

References

- [1] K.R. Baker, *Introduction to Sequencing and Scheduling* (Wiley, New York, 1974).
- [2] S.E. Elmaghraby and P.S. Pulat, Optimal project compression with due-dated events, *Naval Res. Logist. Quart.* 26 (1979) 331-348.
- [3] E.L. Lawler, Optimal sequencing of a single machine subject to precedence constraints, *Management Sci.* 19 (1973) 544-546.
- [4] L.N. Van Wassenhove and L.F. Gelders, Solving a bicriterion scheduling problem, *European J. Oper. Res.* 4 (1980) 42-48.
- [5] R.G. Vickson, Choosing the job sequence and processing times to minimize total processing plus flow cost on a single machine, *Operations Res.* 28 (1980) 1155-1167.
- [6] R.G. Vickson, Two single machine sequencing problems involving controllable job processing times, *AIIE Trans.* 12 (1980) 258-262.